

COSC 4P02 Group 16

Academic Integrity Submission & Similarity Analysis System

User Manual

Project name in interface: Anti-Vibe Coder

Stage 4 Final Documentation

Revision 1.0

April 23, 2026

Revision History

Revision	Date	Author	Description
1.0	April 23, 2026	COSC 4P02 Group 16	Initial Stage 4 user manual generated from the current repository, source code, README, UI routes, API behavior, configuration, and in-application help guides.

Contents

Revision History	1
1 Introduction	4
2 System Overview	4
3 Intended Audience	4
4 Roles and Permissions	5
5 Quick Start	5
5.1 Student: Submit an Assignment	5
5.2 Instructor: Run an Analysis	6
5.3 Instructor: Review a Flagged Pair	6
6 Getting Started	6
6.1 Accessing the Application	6
6.2 Browser Requirements	7
7 Login and Account Access	7
7.1 Student Access	7
7.2 Instructor Sign In	7
7.3 Creating a Professor Account	7
7.4 Account Settings	8
8 Interface Guide	8

8.1	Student Submission Page	8
8.2	Instructor Dashboard	8
8.3	Assignment Workspace	9
8.4	Pair Viewer	9
9	Submission Upload Instructions	9
9.1	Student Single Upload	9
9.2	Bulk Current Upload	9
9.3	Historical Repository Upload	9
9.4	Template Upload	9
9.5	Supported Source Files	9
10	Student Workflow	10
10.1	Access Assignments	10
10.2	Upload a Single Submission	10
10.3	Confirm Upload Status	10
10.4	Repeated Submissions	10
10.5	Bulk Upload for TA / Marker Flow	11
11	Instructor Workflow	11
11.1	Instructor Dashboard	11
11.2	Create an Assignment	11
11.3	Assignment and Offering Management Instructions	12
11.4	Assignment Workspace	12
11.5	Upload Historical Submissions	12
11.6	Upload Template Code	13
11.7	Template Subtraction	13
11.8	Repository Selection	13
11.9	Run Similarity Analysis	14
11.10	View Job Status	14
12	Reviewing Similarity Results	14
12.1	Suspicious Pairs Tab	14
12.2	Similarity Score	15

12.3 Comment Matches	15
13 Side-by-Side Comparison View	15
14 Viewing and Downloading Submissions	16
14.1 Submission Tables	16
14.2 Downloads	16
14.3 Report Export	16
15 Privacy and Academic Integrity Notes	17
16 Common Error Messages and Resolutions	17
17 FAQ	19
18 Troubleshooting Guide	20
19 Administrator and Maintenance-Facing Workflow	21
20 Limitations	22
21 Glossary	23
A Screenshot Placeholders	23

1 Introduction

This user manual explains how to use the Academic Integrity Submission & Similarity Analysis System, implemented in the repository as Anti-Vibe Coder. The system supports academic-integrity review by collecting source-code submissions, preparing uploaded archives, running asynchronous source-code similarity analysis, and presenting instructor-facing evidence for review.

The manual is written for end users. It describes what users can do in the current application and marks unavailable SRS features as **Planned / Not available in this version**. It does not require knowledge of the underlying database, queue, or engine implementation.

2 System Overview

Anti-Vibe Coder provides two main browser-facing areas:

- A public student submission portal at /, where students submit assignment archives using an assignment key.
- An instructor portal at /login and /professor, where professor accounts create assignments, review uploads, upload historical submissions and template code, run comparisons, and inspect suspicious pairs.

The system accepts source archives for Java, C, and C++ assignments. Uploads are prepared asynchronously before they become available for comparison. Similarity analysis is also asynchronous; instructors start comparison runs manually after enough submissions are ready.

The current implementation compares:

- Current submissions against other current submissions.
- Current submissions against historical submissions.

The current implementation does not compare historical submissions against other historical submissions, and template code is treated as reference material rather than as a suspicious submission.

3 Intended Audience

This manual is intended for:

- Students submitting assignment source-code archives.
- Instructors, professors, teaching assistants, or markers reviewing submissions and similarity evidence. The UI labels this role as *Professor* or *Instructor*.
- Deployment or maintenance operators who need to bootstrap the first professor account or check service health.

There is no separate browser-based system administrator interface in the current version.

4 Roles and Permissions

Role	Current Access	Notes
Student / Public Submitter	Can open the public submission page, enter an assignment key, upload a <code>.zip</code> archive, and view the status of that upload while the status token is available in the browser session.	No student login or persistent student history page is exposed in the current web UI. Student identity is encrypted in the browser for normal public submissions.
Professor / Instructor	Can sign in, create assignments, copy assignment keys, upload historical archives and templates, run comparison jobs, review suspicious pairs, reveal identities when available, download prepared archives, update due dates, and manage destructive cleanup actions.	Access is restricted to assignments owned by the signed-in professor account.
System Administrator	Planned / Not available in this version as a browser role.	The repository provides maintenance scripts and health endpoints, but no admin dashboard, user-management screen, or admin RBAC role in the current UI.

Role-based access control (RBAC) is enforced by the API. Professor-only routes reject unauthenticated users and users who do not own the requested assignment.

5 Quick Start

5.1 Student: Submit an Assignment

1. Open the student submission page.
2. Enter your full name, student number, optional email, and the assignment key provided by your instructor.
3. Choose a `.zip` archive containing your assignment source files.
4. Select **Submit Assignment**.
5. Wait for the **Submission Status** panel to show **ready**. If it shows **failed**, read the error and resubmit a corrected archive.

5.2 Instructor: Run an Analysis

1. Sign in through **Instructor Access**.
2. Create an assignment with **New Assignment**, choosing Java, C, or C++.
3. Copy the assignment key from the assignment card and distribute it to students.
4. Wait for current submissions, or upload historical submissions in the assignment workspace under **Uploads**.
5. Optional: upload starter or template code using **Upload Template**.
6. Open the assignment workspace and select **Run Comparison** when the status is **ready to run**, **current**, **stale**, or **failed**.
7. Review results in the **Suspicious Pairs** tab.

5.3 Instructor: Review a Flagged Pair

1. Open an assignment and select the **Suspicious Pairs** tab.
2. Choose **Current vs Current** or **Current vs Historical**.
3. Open a pair from **Suspicious by Code**, **Comment-Supported**, or **Remaining Pairs**.
4. In the pair viewer, inspect the code panes, similarity score, matched regions, matched token count, and comment match count.
5. Use **Code matches**, **Comment matches**, **Back**, and **Next** to navigate evidence spans.
6. Reveal identities only when needed for academic-integrity review.

6 Getting Started

6.1 Accessing the Application

Use the application URL supplied by the institution or deployment operator. In a local deployment, the web application is served by the configured web service, and instructor actions communicate with the API configured by `NEXT_PUBLIC_API_BASE_URL`.

The main navigation includes:

- **Submission Guide**: opens student help from the public submission page.
- **Instructor Access**: opens the instructor sign-in page.
- **Student Portal Access**: returns instructors or authentication pages to the public submission portal.
- **Instructor Guide**: opens instructor help from professor pages.

6.2 Browser Requirements

Normal public student submissions require a modern browser with Web Crypto support. If the browser cannot perform secure encryption, the submission page reports: *This browser does not support secure submission encryption.*

7 Login and Account Access

7.1 Student Access

Students do not sign in in the current public workflow. A student submits by entering:

- Full name.
- Student number.
- Optional email address.
- Assignment key.
- One .zip archive.

The assignment key identifies the assignment. The application normalizes the key by trimming spaces and using lowercase.

7.2 Instructor Sign In

To sign in:

1. Select **Instructor Access**.
2. Enter email and password on the **Instructor Sign In** page.
3. Select **Sign In**.

If credentials are invalid, the application displays *Invalid email or password*. After sign-in, professor users are sent to the **Instructor Dashboard**.

7.3 Creating a Professor Account

The current UI supports professor self-signup:

1. Open `/signup` or select **Create a professor account**.
2. Enter first name, last name, optional title, optional department, email address, password, and password confirmation.
3. Select **Create Account**.

The password must be at least 8 characters and strong enough to pass the UI strength check. Duplicate email addresses are rejected.

7.4 Account Settings

Signed-in professor users can open **Settings** from the instructor dashboard. The settings page supports:

- Viewing the professor account email.
- Changing the password by entering the current password, new password, and confirmation.
- Deleting the account.

Deleting a professor account permanently removes professor access, owned assignments, uploads, and comparison data. Account deletion is blocked while uploads or comparison runs are active.

Password reset by email is **Planned / Not available in this version**.

8 Interface Guide

8.1 Student Submission Page

The student submission page is the first screen at /. It contains:

- Page header labelled **Student Submission** and **Submit Your Assignment**.
- **Submission Form** with identity fields, assignment key, zip upload, and **Submit Assignment**.
- Privacy note explaining browser-side identity encryption.
- **Submission Status** panel after a successful upload request.
- Collapsed **Bulk Upload (TA / Marker Flow)** panel.
- Links to **Submission Guide** and **Instructor Access**.

8.2 Instructor Dashboard

The instructor dashboard at /**professor** contains summary tiles, assignment search, assignment cards, **New Assignment**, **Settings**, and links to the instructor guide. Assignment cards show the assignment language, active key, due date when present, current and historical submission counts, latest pair count when present, and comparison status.

8.3 Assignment Workspace

The assignment workspace is a full-screen view for one assignment. Its sidebar contains **Submissions**, **Suspicious Pairs**, **Uploads**, and **Danger Zone**. The header contains **Back to Dashboard**, **Instructor Guide**, **Student Portal Access**, the active assignment key, optional due date, **Download**, and **Run Comparison**.

8.4 Pair Viewer

The pair viewer opens from a suspicious pair row. It includes pair-level metrics, identity panels, read-only side-by-side source panes, and match navigation controls. The **Back to Sus Pairs** link returns to the assignment workspace results tab.

9 Submission Upload Instructions

9.1 Student Single Upload

Use this flow when one student is submitting one assignment archive. The uploaded archive must be a **.zip**. The archive may contain folders, nested zips, and source files, but only source files matching the assignment language are prepared for analysis.

9.2 Bulk Current Upload

Use this flow when a TA or marker is uploading multiple current submissions at once from the public page. The parent zip must contain child submission zips directly at the top level or inside one top-level wrapper folder. Each child zip becomes one current submission.

9.3 Historical Repository Upload

Use this instructor flow from the **Uploads** tab when adding prior submissions for current-vs-historical comparison. The parent zip must follow the same child-zip structure used by bulk current upload.

9.4 Template Upload

Use this instructor flow from the **Uploads** tab when adding starter or reference code. Template upload expects one **.zip** archive and creates one template version.

9.5 Supported Source Files

Only files matching the selected assignment language are prepared:

- Java: **.java**.

- C: .c, .h.
- C++: .cpp, .cc, .cxx, .hpp, .hh, .hxx, .h.

10 Student Workflow

10.1 Access Assignments

Students access an assignment by receiving an assignment key from the instructor. The current student portal does not list available assignments and does not show an assignment title before upload.

10.2 Upload a Single Submission

1. Open the public submission page.
2. Complete required fields: **Full Name**, **Student Number**, and **Assignment Key**.
3. Optionally enter **Email**.
4. Drag a .zip file into the upload area or click to browse.
5. Select **Submit Assignment**.

The **Submit Assignment** button remains disabled until the required identity fields, assignment key, and archive are present. During upload, the button text changes to **Encrypting & Uploading...**

10.3 Confirm Upload Status

After upload, the page displays **Submission Status**. The status panel can show:

- **received**: the upload was accepted and is waiting for preparation.
- **processing**: the archive is being prepared.
- **ready**: source files were prepared successfully.
- **failed**: the archive could not be prepared.

The public page checks status using a temporary status token returned after upload. If the page is closed or refreshed in a way that loses that token, the current version does not provide a student login page to retrieve previous submission history.

10.4 Repeated Submissions

If the same student submits again, the system creates a separate current submission entry. Instructors see each prepared upload independently.

10.5 Bulk Upload for TA / Marker Flow

The public submission page also includes a collapsed section named **Bulk Upload (TA / Marker Flow)**. This flow uploads one parent zip containing child zip files, with one child zip per submission. Each child zip filename stem becomes the visible submission name.

Bulk upload should be used carefully:

- Do not include student names in child zip filenames unless those names may be visible to the instructor.
- Child zip filenames must be unique after sanitization.
- Child zips with no supported source files may be skipped, and the status panel may show a warning.

11 Instructor Workflow

11.1 Instructor Dashboard

After sign-in, the **Instructor Dashboard** shows:

- Summary tiles for assignments, current submissions, historical submissions, and detected pairs.
- A search box for assignment title, language, or active assignment key.
- Assignment cards with language, comparison status, assignment key, due date, submission counts, pair count, and actions.
- **New Assignment**, **View Details**, **Edit Due Date**, and **Settings**.

11.2 Create an Assignment

1. Select **New Assignment**.
2. Enter an **Assignment Title**.
3. Select a **Language**: Java, C, or C++.
4. Optionally set a **Due Date**.
5. Select **Create Assignment**.

The system generates an active assignment key. Copy this key from the assignment card and provide it to students.

Due dates are informational only in the current version. They are displayed in the instructor UI and do not block late uploads.

11.3 Assignment and Offering Management Instructions

The current implementation manages assignments directly. Assignment actions include:

- Search assignments on the dashboard.
- Copy assignment keys from assignment cards.
- Open an assignment with **View Details**.
- Edit, clear, or set due dates.
- Delete an assignment from the assignment workspace **Danger Zone**.

Offering management is **Planned / Not available in this version**. The current repository does not expose offering records, offering pages, section management, roster management, or cross-offering assignment grouping.

11.4 Assignment Workspace

The assignment workspace is opened with **View Details**. It contains a header with the assignment title, language, active key, due date, comparison status, **Download**, and **Run Comparison**. The sidebar contains:

- **Submissions**
- **Suspicious Pairs**
- **Uploads**
- **Danger Zone**

11.5 Upload Historical Submissions

Historical submissions are used as a repository for current-vs-historical comparison.

1. Open the assignment workspace.
2. Select the **Uploads** tab.
3. Under **Historical Submissions**, choose a parent **.zip** archive.
4. Select **Upload Historical**.
5. Wait for the upload status to become **ready**.

Accepted historical archive structures are:

- Parent zip containing child submission zips directly at the top level.

- Parent zip containing one top-level wrapper folder, which contains child submission zips.

Only first-layer child zip files define historical submissions. Historical-vs-historical comparison is not performed.

11.6 Upload Template Code

Template code represents starter, reference, or provided code that should not count as suspicious student similarity.

1. Open the assignment workspace.
2. Select **Uploads**.
3. Under **Template Code**, choose a **.zip** archive.
4. Select **Upload Template**.
5. Wait for the template upload to finish.

Each successful template upload creates a new template version. The newest uploaded template is active. Template versions can be viewed, downloaded, or deleted from the **Submissions** tab.

11.7 Template Subtraction

Template subtraction is implemented as template masking during similarity analysis. When an active template exists, the comparison engine parses the template and masks sufficiently long matches between a submission and the template. Masked template regions are excluded from the code similarity score so provided starter code does not inflate student-to-student similarity.

Important behavior:

- Templates are not displayed as suspicious submissions.
- Uploading, deleting, or changing templates makes previous comparison results stale or clears comparison data, depending on the action.
- Instructors should rerun comparison after changing template material.

11.8 Repository Selection

The SRS describes repository selection for cross-offering comparison. In the current implementation there is no managed repository catalogue or repository-selection screen. The implemented repository-like workflow is historical upload:

- Instructors select a historical parent zip from their local machine.
- The system prepares its child zips as historical submissions.

- Comparison results include **Current vs Historical** pairs.

Selecting from stored offerings, institutional repositories, or previous-course repositories is **Planned** / **Not available in this version**.

11.9 Run Similarity Analysis

Comparison runs are manual.

1. Confirm upload preparation has completed.
2. Confirm the assignment has at least one current submission and at least two non-template submissions total.
3. Select **Run Comparison**.
4. Wait for the status to move through queued/running and finish.

If uploads are still processing, the run is blocked. If another run is already queued or running, a second run is blocked.

11.10 View Job Status

The dashboard and assignment workspace show comparison status:

- **not ready**: not enough prepared non-template submissions.
- **preparing**: uploads are still being prepared.
- **ready to run**: prepared inputs are ready and no current result exists.
- **running**: a comparison is queued or running.
- **current**: current results match the latest prepared inputs.
- **stale**: inputs changed after the last completed run.
- **failed**: the latest comparison attempt failed.

Upload batches show statuses such as **received**, **processing**, **ready**, and **failed**. Upload history is available in the assignment workspace **Uploads** tab.

12 Reviewing Similarity Results

12.1 Suspicious Pairs Tab

Open **Suspicious Pairs** from the assignment workspace after a comparison run completes. Results are divided into pair scopes:

- **Current vs Current**
- **Current vs Historical**

Within each scope, the system groups pairs into:

- **Suspicious by Code**: code similarity of at least 35.0%.
- **Comment-Supported**: lower code similarity but at least one meaningful comment match.
- **Remaining Pairs**: shown only after selecting **Show all pairs**.

The table displays left submission, right submission, code similarity, comment match count, total match count, and an **Open** action.

Manual column sorting and arbitrary filtering are **Planned / Not available in this version**. The current UI uses the built-in grouping, priority ordering, pair type switch, and **Show all pairs** control.

12.2 Similarity Score

The code similarity score is calculated from normalized code token matches after template masking. It is a review signal, not a misconduct decision.

12.3 Comment Matches

Comment matches are supporting evidence only. They do not affect the numeric code similarity score. The interface displays a comment match count rather than a separate comment similarity score.

13 Side-by-Side Comparison View

Open a pair with **Open**. The pair viewer shows:

- Assignment name and language.
- The two submission display names.
- Similarity percentage.
- Number of matched regions.
- Matched token count.
- Comment match count.
- Identity panels for each side.
- Read-only side-by-side code panes.

- **Code matches** and **Comment matches** navigation.

Use the evidence viewer as follows:

- Select a numbered match button to scroll both panes to the corresponding evidence span.
- Use **Back** and **Next** within **Code matches** or **Comment matches**.
- Click a highlighted region in either code pane to activate its matching region on the other side.
- Use **Back to Sus Pairs** to return to the assignment results tab.

Colors stay matched across both panes for the same evidence span. Only one match is active at a time.

14 Viewing and Downloading Submissions

14.1 Submission Tables

The **Submissions** tab shows current submissions, historical submissions, and template versions. For each submission, the table shows display name, file count, upload time, and action buttons.

Use the eye icon to view a submission inline. The detail viewer shows prepared source files and allows identity reveal when available.

14.2 Downloads

The current implementation supports archive downloads:

- **Download All:** downloads all current submissions for the assignment.
- Submission row **Download:** downloads one prepared submission archive.
- Template row **Download:** downloads one prepared template archive.

14.3 Report Export

Formal report export, such as PDF or CSV export of similarity results, is **Planned / Not available in this version**. The current UI does not provide a dedicated **Export Report** action for pair tables or side-by-side evidence. Instructors can review evidence online and download prepared archives, but report generation must be handled outside the application in this version.

15 Privacy and Academic Integrity Notes

The system is intended to support academic-integrity review. It does not automatically determine misconduct.

Privacy-related behavior in the current implementation includes:

- Normal public student submissions encrypt identity data in the browser before upload.
- Instructors see alias-style display names such as SUB-... by default for encrypted public submissions.
- Instructors can use **Reveal Identity** when identity review is necessary.
- Bulk public upload uses sanitized child zip filename stems as visible display names. These names are not encrypted identities.
- Professor assignment access is restricted by role and ownership.
- Status access for public uploads uses a status token returned after upload.

Similarity scores, evidence spans, and comment matches should be reviewed with context. The system should be used as a decision-support tool for instructors and academic-integrity processes, not as an automatic accusation or disciplinary decision.

16 Common Error Messages and Resolutions

Message	Meaning	Resolution
Invalid email or password	The instructor credentials did not match an account.	Re-enter credentials. If no account exists, create a professor account or use the bootstrapped account supplied by the deployment operator.
That assignment key is invalid or inactive	The student entered an incorrect assignment key, or the key is not active.	Copy the key exactly from the instructor assignment card. Remove extra spaces.
uploaded archive must be a zip file	The selected file was not a .zip.	Compress the source folder as a zip archive and upload again.
We couldn't read that zip archive. Please upload a valid zip file.	The uploaded file could not be extracted as a zip.	Recreate the archive and confirm it opens locally before resubmitting.

No supported source files were found in that archive for this assignment language.	The archive did not contain files matching the assignment language.	Upload files with supported extensions: <code>.java</code> , <code>.c</code> , <code>.h</code> , <code>.cpp</code> , <code>.cc</code> , <code>.cxx</code> , <code>.hpp</code> , <code>.hh</code> , or <code>.hxx</code> , depending on the assignment language.
That bulk archive must contain direct child zip files, or one top-level folder containing child zip files, with one child zip per submission.	The bulk current upload shape is invalid.	Repackage the parent zip so each first-layer child zip represents one submission.
That historical archive must contain child zip files either directly at the top level or inside one top-level folder.	The historical upload shape is invalid.	Repackage the historical archive using the supported parent-zip structure.
That archive is too deeply nested to process.	The archive exceeded the configured nesting-depth limit.	Flatten the archive structure and upload again.
That archive contains too many source files to process.	The archive exceeded the configured source-file limit.	Remove unrelated files and resubmit only assignment source files.
That archive is too large to process after extraction.	The archive exceeded the expanded-size limit.	Remove build output, dependencies, and generated files; then upload again.
Uploads are still being prepared for this assignment. Please wait for them to finish before running comparisons.	A comparison was requested before upload preparation completed.	Wait for upload status ready , then select Run Comparison again.
A comparison run is already queued or running for this assignment.	Another comparison job is active.	Wait for the active run to finish.
A comparison run requires at least one current submission.	There are no current submissions.	Wait for students to submit or use the bulk current upload flow.
At least two non-template submissions are required to run a comparison.	There are not enough current plus historical submissions.	Add another current submission or upload historical submissions.
This assignment has uploads or comparison runs in progress. Please wait for them to finish before deleting anything.	A destructive action was attempted while jobs were active.	Wait for active uploads or comparisons to complete before deleting.

17 FAQ

1. Do students need an account?

No. The current student workflow is public and uses an assignment key. There is no student login or student history page in the current UI.

2. How does a student know which assignment to submit to?

The instructor provides the assignment key. The public submission page does not list assignments.

3. What file format should students upload?

Students should upload one .zip archive containing their assignment source files.

4. Which programming languages are supported?

Assignments can be created for Java, C, or C++. Java accepts .java; C accepts .c and .h; C++ accepts .cpp, .cc, .cxx, .hpp, .hh, .hxx, and .h.

5. Are comparisons automatic after upload?

No. Uploads are prepared automatically, but the instructor starts comparisons manually with **Run Comparison**.

6. How many submissions are needed before analysis can run?

At least one current submission is required, and there must be at least two non-template submissions total.

7. What does template upload do?

Template upload adds starter or reference code. Active template code is masked during analysis so it does not inflate student-to-student similarity scores.

8. Can an instructor compare against previous terms or repositories?

The current implementation supports current-vs-historical comparison by uploading historical submissions as a parent zip. A managed repository-selection or cross-offering browser is not available in this version.

9. What does the 35% threshold mean?

Pairs with code similarity of at least 35.0% appear under **Suspicious by Code**. This threshold is a review aid, not a misconduct finding.

10. Do comment matches affect similarity score?

No. Comment matches are displayed as supporting evidence and do not affect the code similarity score.

11. Can instructors export a formal report?

Formal report export is not implemented. Instructors can review results online and download prepared submission or template archives.

12. Can an instructor reveal student identity?

Yes, when identity is available. Normal public submissions store encrypted identity information and expose a **Reveal Identity** action to the professor. Bulk public submissions use display names derived from child zip filenames.

13. Why are results marked stale?

Results become stale when assignment inputs change after the last completed run, such as when submissions or templates are added or changed. Rerun comparison for current results.

14. Can instructors delete data?

Yes. The **Danger Zone** supports deleting one validated submission, deleting all items in a category, and deleting the entire assignment. These actions are irreversible and clear comparison data.

18 Troubleshooting Guide

Problem	Likely Cause	Recommended Action
Student cannot submit because the button is disabled.	Required fields or archive are missing.	Enter full name, student number, assignment key, and select a .zip file.
Student upload fails immediately.	The assignment key is wrong or the file is not a zip archive.	Confirm the assignment key with the instructor and upload a valid .zip .

Upload remains in processing.	Worker processing is still running or delayed.	Wait and refresh status. If it remains stuck, contact the deployment operator.
Upload fails after processing.	The archive has no supported source files, has invalid structure, is too large, or exceeds limits.	Repackage only relevant source files using the supported archive structure.
Bulk upload creates unexpected visible names.	Child zip filename stems become submission display names.	Rename child zip files before upload. Avoid student names unless intentionally visible.
Instructor cannot run comparison.	Assignment is not ready, uploads are active, or a run is already active.	Wait for uploads to finish and ensure enough non-template submissions exist.
No pairs appear after running analysis.	The selected pair scope has no results, or all pairs are lower-priority and hidden.	Switch between Current vs Current and Current vs Historical ; use Show all pairs when available.
Results are stale.	Inputs changed after the last completed comparison.	Select Run Comparison again.
Identity cannot be revealed.	The submission has no encrypted identity or revealable display-name mode.	Use the visible display name and available evidence. For future bulk uploads, choose filenames carefully.
Account deletion is blocked.	One or more uploads or comparisons are active.	Wait for active jobs to complete, then retry from Settings .
Downloaded archive is missing unrelated files.	The system stores prepared source files, not every uploaded file.	Use original local archives if non-source files are required. The application downloads prepared source artifacts.
Formal report export is missing.	Report export is not implemented.	Review evidence online and download archives as needed.

19 Administrator and Maintenance-Facing Workflow

There is no browser-based administrator workflow in the current version. The repository supports the following maintenance-facing operations:

- Bootstrap the first professor account using the deployment environment variables `ADMIN_BOOTSTRAP_EMAIL` and `ADMIN_BOOTSTRAP_PASSWORD`, then running `npm run db:bootstrap-admin`.
- Check API liveness at `/health/live`.

- Check API readiness at `/health/ready`, which verifies database and Redis connectivity.
- Use deployment configuration to provide PostgreSQL, Redis, filesystem-backed object storage, the worker service, and the Rust engine binary.

These tasks are operational tasks, not end-user UI actions. System administrator RBAC, browser-based user management, audit dashboards, and institutional roster administration are **Planned / Not available in this version**.

20 Limitations

The following limitations were identified from the current repository:

- No separate browser-based system administrator role or admin dashboard is implemented.
- No offering management UI or offering data model is exposed.
- No managed repository-selection catalogue is implemented; historical upload is the available repository-like workflow.
- No student login, persistent student submission history page, or student account recovery flow is exposed in the web UI.
- No email verification or password reset flow is implemented.
- Formal PDF/CSV report export is not implemented.
- Result tables are grouped and prioritized, but arbitrary column sorting and filtering are not implemented.
- Due dates are informational and do not block uploads.
- The UI accepts zip archives; other archive formats are not supported.
- Only Java, C, and C++ assignment languages are supported.
- Only files matching supported source extensions are prepared and analyzed.
- Prepared downloads contain stored source artifacts, not necessarily every original uploaded file.
- Filesystem-backed object storage is the supported storage mode in the current deployment configuration.
- Public upload status is available through a returned token; there is no later public lookup form.

21 Glossary

Term	Definition
Submission	A prepared student or historical source-code artifact associated with an assignment. Current submissions come from student or bulk current uploads; historical submissions come from historical archives.
Template	Starter, reference, or provided code uploaded by an instructor. Templates are versioned, and the active template is used for masking during analysis.
Repository	In the SRS, a collection of prior submissions for comparison. In the current implementation, this is represented by uploading historical submissions as a parent zip archive.
Offering	A course section, term, or delivery instance described by the SRS. Offering management is not available in the current implementation.
Analysis Job	An asynchronous comparison run queued by selecting Run Comparison . It processes prepared submissions and produces pair results.
Similarity Score	The numeric code similarity value shown for a pair, displayed as a percentage. It is based on matched normalized code tokens after template masking.
Evidence Span	A matched region in the left and right submission shown in the side-by-side evidence viewer. Evidence spans may be code matches or comment matches.
Template Subtraction	The system behavior that masks active template code so provided code does not inflate student-to-student similarity scores.
Anonymized Identifier	A display name, such as SUB- . . . , shown instead of raw student identity for encrypted public submissions.
RBAC	Role-based access control. The API restricts professor-only functionality and assignment ownership.

A Screenshot Placeholders

The following figures are placeholders for final binder screenshots. Replace each placeholder with an actual screenshot during final document assembly if screenshots are required by the submission format.

Submit Your Assignment

Enter your details, provide the assignment key, and upload your zip.
Your identity is encrypted in your browser before anything is sent to the server.

[Instructor Access](#)

Submission Form

Full Name *	Student Number *
Jane Student	1234567

Email (optional)

Assignment Key *

Zip Archive *



Drop your .zip file here
or click to browse

[Submit Assignment](#)

 Your name, student number, and email are encrypted using your institution's public key before submission. Only the instructor can decrypt your identity.

Figure 1: Student submission page.

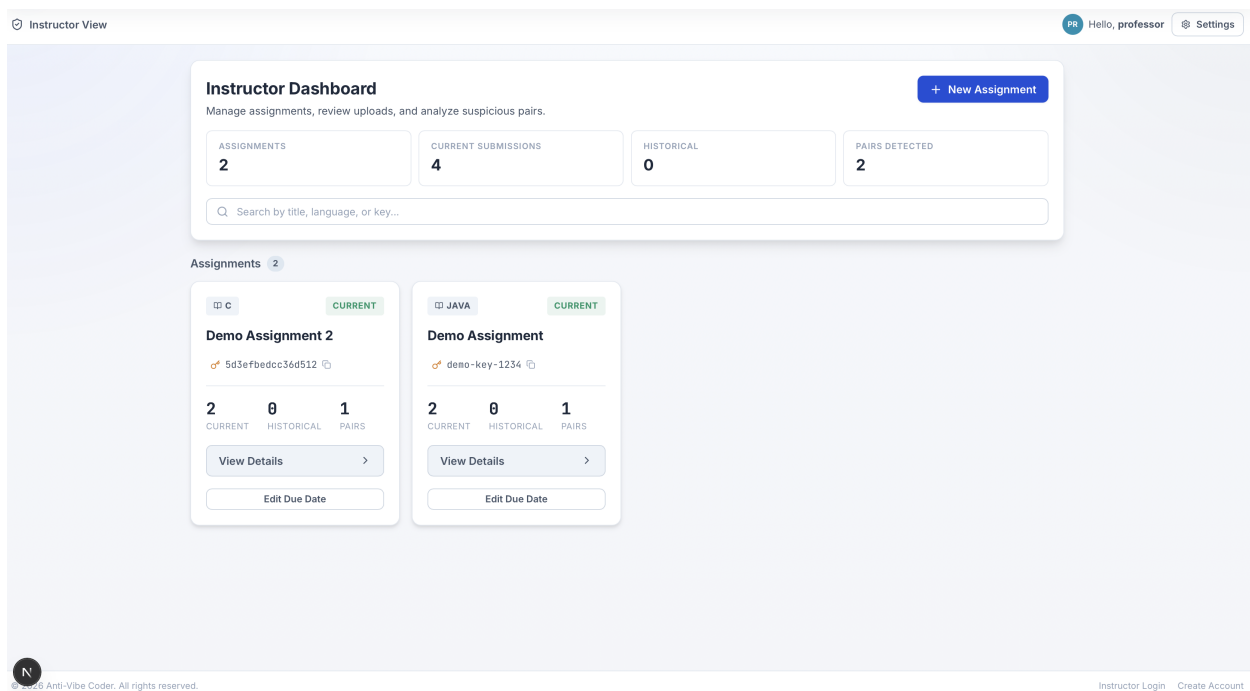


Figure 2: Instructor dashboard.

CURRENT
2

HISTORICAL
0

TEMPLATES
0

Current Submissions **2**

Download All

NAME	FILES	UPLOADED	ACTIONS
SUB-B73AD6629A	1	2026-04-24, 11:40:57 AM	
SUB-5780751C7C	1	2026-04-24, 11:41:07 AM	

Historical Submissions **0**

No historical submissions yet. Upload historical archives in the Uploads tab.

Template Versions **0**

No template uploaded yet. Upload starter code in the Uploads tab.

Figure 3: Assignment workspace submissions tab.

Student Submission

READY

This upload finished successfully.

Historical Submissions

Upload one parent archive. First-layer child zip files become historical submissions.

Zip Archive

Choose File no file selected

Upload Historical

Template Code

Upload starter or provided code. Template files stay separate from pair review.

Zip Archive

Choose File no file selected

Upload Template

Due Date

No due date set.

Set Due Date

Upload History

Student Submission

2026-04-24, 11:41:07 AM

READY

Student Submission

2026-04-24, 11:40:57 AM

READY

Figure 4: Assignment workspace uploads tab.

Comparison Results

CURRENT Current results match the latest prepared inputs.
 Run Comparison

Current vs Current

Current vs Historical

Showing 1 of 1 pairs

Suspicious by Code

Code similarity $\geq 35.0\%$ 1

LEFT	RIGHT	CODE SIM	COMMENTS	MATCHES	
SUB-B73AD6629A Current	SUB-5780751C7C Current	100.0%	2	3	<button>Open</button>

Figure 5: Suspicious pairs review tab.

SUB-B73AD6629A vs SUB-5780751C7C
 Demo Assignment 2 - C

100% similar
3 regions
9090 tokens

2 comments

Comment matches do not affect the code similarity score. They are only supporting evidence, such as when copied code is heavily refactored but one or more long comments still match.

SUB-B73AD6629A CURRENT

Reveal Identity

This submission stores an encrypted identity that can be revealed by the professor.

SUB-5780751C7C CURRENT

Reveal Identity

This submission stores an encrypted identity that can be revealed by the professor.

SUB-B73AD6629A

current - click a highlighted region to jump to its pair

```

1  /* Live demo C submission: intentionally large generated test
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <stdint.h>
6
7  typedef struct DemoRecord {
8      int id;
9      int category;
10     long value;
11     char label[32];
12 } DemoRecord;
13
14 static void alex_initialize_records(DemoRecord *records, int c
15     for (int i = 0; i < count; ++i) {
16         records[i].id = i + 1;
17         records[i].category = (i * 3 + 7) % 17;
18         records[i].value = (long)(i + 1) * 41;
19         snprintf(records[i].label, sizeof(records[i].label), "
20     }
21 }
22
23 static long alex_prefix_0(DemoRecord *records, int count) {

```

SUB-5780751C7C

current - colors stay matched on both sides

```

1  /* Live demo C submission: intentionally large generated test
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <stdint.h>
6
7  typedef struct DemoRecord {
8      int id;
9      int category;
10     long value;
11     char label[32];
12 } DemoRecord;
13
14 static void blake_initialize_records(DemoRecord *records, int
15     for (int i = 0; i < count; ++i) {
16         records[i].id = i + 1;
17         records[i].category = (i * 5 + 11) % 17;
18         records[i].value = (long)(i + 1) * 53;
19         snprintf(records[i].label, sizeof(records[i].label), "
20     }
21 }
22
23 static long blake_prefix_0(DemoRecord *records, int count) {

```

Figure 6: Side-by-side pair comparison view.

Account Settings

Manage your professor account, credentials, and account deletion.



professor@example.com
PROFESSOR ACCOUNT

Change Password

Current Password

.....

New Password

Choose a new password

Confirm New Password

Repeat new password

Update Password

Danger Zone

Deleting your account permanently removes your professor access, all owned assignments, uploads, and comparison data.

Delete Account

Figure 7: Professor account settings page.